

SMARTRABBIT: An Interactive Query Processor

Anonymous Author(s)

ABSTRACT

Traditional relational database systems optimize analytical queries to minimize their end-to-end latency. The resulting optimal plans are usually *blocking*, forcing users to wait until full query completion before seeing any results. This execution model precludes *interactivity*, i.e., users cannot observe partial results or gain early insights for long-running queries. Query optimizers rarely choose plans that promote interactivity, since such plans either incur prohibitively large latencies or involve operators for which interactive alternatives are unavailable. This paper introduces a first-of-its-kind novel interactive query processor named SMARTRABBIT that promotes interactivity of answers while matching the end-to-end latency of blocking execution plans. We achieve this by first designing a plan optimized for interactivity for a given query, and then simultaneously executing this plan alongside a traditional blocking plan. The two executions are carefully synchronized to maintain the correct order of answers and prevent duplicates. We implement SMARTRABBIT in a scalable, open-source database system and show that SMARTRABBIT consistently delivers early and continuous results across various analytical queries, data scales, and parallel (multi-node, multi-partition) system instances, with only minimal latency overhead compared to traditional blocking execution.

CCS CONCEPTS

• Do Not Use This Code → Generate the Correct Terms.

KEYWORDS

Interactivity, Query Processing, Query Optimization

1 INTRODUCTION

Relational database systems typically optimize and process large analytical queries with the objective of minimizing end-to-end latencies, i.e., the time between arrival of the query to returning the full answer set [13, 29]. The chosen execution plans often result in blocked execution in which the system performs a series of non-pipelined operations (such as sorting or hashing) during which no output is produced [26]. The system then outputs all the answers at the end of the execution. As an example, an optimized TPC-H Q3 at a scale factor 300, running on a 80 core machine on a modern multi-node database such as AsterixDB could take tens of minutes to complete. The user (or the downstream processor) sits idle for the duration, before all the answers arrive at the end. Contrast the above execution with an interactive execution that continuously

produces outputs incrementally throughout its execution. For some queries, incremental execution could be achieved using interactive alternatives to operators, such as (index) nested-loop instead of hash or sort-merge for executing joins. Such plans typically result in unacceptable increase in end-to-end latency – the Q3 above may now take hours instead of minutes – and, hence, are never chosen by the optimizer. Furthermore, current systems do not support ways for incremental sorting or aggregation, and, hence, are unable to execute queries with group-by and order-by incrementally.

Interactivity during data analysis can be important for several reasons. When analysis queries last several minutes to hours, access to partial/intermediate results could provide initial insights, reveal useful patterns and guide further exploration [15]. Analysts frequently engage in multistage analysis where the result of one query informs the next in a sequence of exploratory or hypothesis-driven steps. Early and continuous result delivery in such workflows could shorten the feedback loop, improve resource utilization by supporting pipelined execution across analysis stages, and help steer computation toward more promising directions [11]. Interactivity is also useful in top-K aggregations that arise in analytical dashboards, recommendation systems, scientific exploration, and monitoring pipelines [17, 18] and also recently in intermediate stages of LLM answering [20].

Our goal in this work is to develop a general framework for interactive query answering that generates answers incrementally terminating in the generation of the exact answer set while maintaining latency comparable to the default execution of the blocking plan generated by optimizing end-to-end latency. Towards the above goal, we develop a general-purpose query executor SMARTRABBIT that continuously returns answers to users interactively until all answers have been returned. The key idea behind SMARTRABBIT is that while the traditional blocking execution proceeds in the background, we simultaneously initiate a second execution path that is optimized for interactivity and leverages system resources left idle by the blocking plan. Given a query, SMARTRABBIT employs a rule-based framework to derive an interactive plan alongside the conventional blocking one. Both executions run in parallel: the interactive execution begins producing early results eagerly, whereas the blocking execution works in the background to eventually produce the complete result set. When the blocking execution is ready to produce answers, the interactive execution is stopped, and the blocking execution takes over and produces all the remaining answers that were not yet produced by the interactive execution. Note that unlike continuous or streaming query systems, SMARTRABBIT operates on static datasets and produces the complete query result, but does so interactively.

Given its importance in analysis, several prior approaches in the literature have explored ways to improve interactivity. Two such examples are the LIMIT clause in SQL and approximate query processing (AQP). LIMIT clause, pioneered in [9], empowers users to specify the number of answers they seek to retrieve as part of the query. Given a LIMIT query, the optimizer may choose a

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

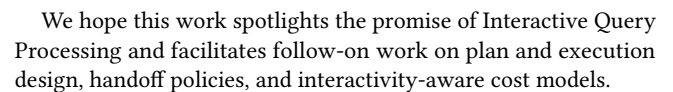
SIGMOD '26, May 31 - June 5, 2026, Bengaluru, India

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/18/06

<https://doi.org/XXXXXXX.XXXXXXX>

Figure 1 shows the performance of SMARTRABBIT compared to a pure blocking approach on TPC-H Q3 on a scale factor of 300. As we can see, SMARTRABBIT starts producing answers almost instantly and continues producing answers from the interactive execution until the blocking execution takes over. It ends up producing about 60% of the answers before the blocking execution takes over and produces the remaining answers. On the other hand, in the pure blocking execution, the user waits for half an hour before seeing any answer. This clearly shows the ability of SMARTRABBIT to produce answers interactively at only a minimal latency overhead. We



2

2 OPERATOR INTERACTIVITY

In order to understand how to answer a query interactively, we need to understand how to design each operator of the query interactively. Some operators, such as index scans, can produce results almost immediately, while others, like aggregation without grouping, require all input tuples before yielding any output. To capture this spectrum, we define two temporal markers for every operator: the moment it can first emit an output, and the moment it is guaranteed to have produced all outputs. We refer to these as the *inception point* and *completion point* of the operator respectively.

2.1 Inception Point and Completion Point

Definition 2.1 (Inception Point). The *inception point* (IP) of an operator is the earliest time it can produce any output after its first input arrives.

Definition 2.2 (Completion Point). The *completion point* (CP) of an operator is the earliest time at which the operator has produced all of its output tuples after its first input arrives.

While inception and completion points characterize the interactive potential of individual operators, query execution involves multiple operators typically connected in a directed acyclic graph [13]. Let p and c denote two operators such that c consumes the output of p ; we refer to p as the *producer* (or upstream) operator and c as the *child* (or downstream) operator. An operator's ability to produce results thus depends not only on its own behavior but also on when its inputs become available from its producer(s). This motivates a notion of *blocking* between operators, which captures how dependencies propagate interactivity (or lack thereof) across the plan. We now present the following definitions and lemmas.

Definition 2.3 (Blocking Operator). For an edge $p \rightarrow c$ where c consumes the output of p , we say that c is *blocking on p* if c cannot produce any output until p has fully completed.

LEMMA 2.1 (BLOCKING CHARACTERIZATION). For an edge $p \rightarrow c$ between operators, c is *blocking on p* if and only if

$$ip_c \geq cp_p.$$

LEMMA 2.2 (INTERACTIVE CHARACTERIZATION). For an edge $p \rightarrow c$, c is *interactive on p* if and only if

$$ip_c < cp_p.$$

Definition 2.4 (Interactive Operator Implementation). An operator implementation with multiple input operators $\{p_1, p_2, \dots, p_k\}$ is *interactive* if it is interactive on every input operator, i.e.,

$$ip_c < cp_{p_i} \quad \text{for all } i = 1, \dots, k.$$

Thus, any operator implementation that can start producing outputs without waiting for any of its input operators to complete qualifies as an *interactive operator*. We use this principle to distinguish interactive and blocking implementations of different operators.

2.2 Inception Point and Completion Point of Different Operators

2.2.1 Selection and Projection. Selection and projection are inherently interactive because no implementation of them requires full

knowledge of the input before producing output. In a system that pipelines its operators, a selection only evaluates a local predicate on each incoming tuple (or vector of tuples in a block-oriented engine) and can forward qualifying tuples immediately. Similarly, a projection simply rewrites or discards attributes, again producing tuple- or block-at-a-time, without waiting for the entire input. Consequently, under any physical implementation in pipelining systems - these operators produce their first outputs strictly before the producer's completion point, and are therefore interactive.

2.2.2 Joins. The inception point of a join operator is the first time any join result can be produced, and its completion point is when all join results have been produced. An interactive join operator implementation is one which does not require *any* of its joining relations to produce all their tuples before it can produce a join result. Let us look at the different types of join implementations. In this subsection, R and S denote the left and right inputs of a join operator, respectively.

Nested-loop join (NLJ). In a nested loop join, we take a tuple (or a block) of the outer relation (R), and scan the full inner relation (S). A nested loop join operator is potentially ready to produce its first output after it has scanned a tuple (or a block) of the outer and the first tuple from the inner (whether it finds a valid join result is a separate issue). Hence Nested Loop Join is an interactive implementation as it is not blocking on either of its inputs and its inception point can be as low as the maximum of the inception points of both the joining relations:

$$ip_{NLJ} \geq \max(ip_R, ip_S).$$

Index nested-loop join (INLJ). In an INLJ, we take a tuple (or a block) of the outer relation and probe an index on the join key of the inner. Since the index is immediately available, the join can produce results as soon as the outer yields its first tuple. Thus the inception point is bounded by the outer relation:

$$ip_{INLJ} \geq ip_R.$$

Because the index provides direct access to matching tuples in S without scanning it entirely, INLJ can produce qualifying join results earlier than a block-oriented NLJ.

Hash join (build on R). In the classical build-probe variant of a hash join, the join must first consume the entire build input R to construct the hash table. Thus it is blocking on R , with

$$ip_{HJ} \geq cp_R,$$

and no output can be produced until R is complete. By contrast, *hybrid hash join* [13] (HHJ) keeps one partition of R (say R_0) in memory while writing the others to disk. Once R_0 is fully materialized, the join can begin producing results for tuples from L that hash into the same partition. Formally, its inception point can be as low as

$$ip_{HHJ} \geq \max(ip_L, cp_{R_0}),$$

with $cp_{R_0} \leq cp_R$. Hence HHJ may emit results before the entirety of R is read, but typically later than NLJ or INLJ.

Sort-merge join (SMJ). In the typical implementation, SMJ first sorts both inputs on the join key and then performs a merge phase to generate join results. Because the sort phase is blocking, the join

cannot produce any output until both inputs are fully sorted. Thus SMJ is blocking on *both inputs R and S*, with

$$ip_{SMJ} \geq \max(cp_R, cp_S).$$

Progressive/Overlapped joins in literature (Ripple Join, XJoin, HMJ, Progressive Merge Join). These explicitly reduce blocking by overlapping build/probe or interleaving scans, ensuring $ip_{join} < cp_R$ and $ip_{join} < cp_S$ under typical conditions, i.e., earlier inception point than classical hash or sort-merge join [10, 14, 24, 33].

2.2.3 Aggregation. Aggregation operators combine multiple input tuples into one or more result tuples. For a given group g , the group's *inception point* ip_g is the time its first member arrives and its *completion point* cp_g is when the last member arrives from the upstream operator. At the *operator level*, the inception point ip_{agg} denotes the earliest time any group g can emit its aggregate.

Without GROUP BY. If no GROUP BY is present, aggregation is global, and the operator's inception point is bounded by the completion point of its upstream operator p .

$$ip_{agg} \geq cp_p,$$

so global aggregation is inherently blocking.

With GROUP BY. When grouping is present, each group g has its own cp_g , and the operator can emit results as soon as a group completes. In principle, this allows interactive execution, but the most common implementations are blocking:

Sort-based aggregation. Tuples are sorted on the grouping key and then scanned to produce group aggregates. Because the sort phase is blocking, the operator cannot produce any output until sorting completes. Hence sort-based aggregation is blocking, with

$$ip_{agg} \geq cp_{sort}.$$

Hash-based aggregation. Tuples are inserted into buckets keyed by the grouping attribute. While partial aggregates can be updated incrementally, the operator cannot guarantee group completion until all input tuples have been processed. Thus hash-based aggregation is blocking, with

$$ip_{agg} \geq cp_p.$$

We introduce two non-blocking aggregation variants below.

Index-based grouping. If an index exists on the grouping attribute, scanning the index yields tuples in group order. Once all tuples for a key have been seen, its aggregate can be emitted immediately. Hence, index-based grouping is interactive, with

$$ip_{agg} = \min_g cp_g < cp_p.$$

This idea of index-driven grouping was first discussed in [15]. Note that even in the absence of an explicit index on the grouping key(s), if the incoming tuples appear in a *sorted order*, we can still do interactive grouping with the same principle: once we see all tuples with the same grouping key, and the grouping key tuple changes, we are ready to output the aggregate value for that group.

Shuffle on grouping key. A *shuffle* (or *exchange*) operator [12] is commonly used in parallel query engines to redistribute tuples across workers based on a partitioning key. We can consider shuffling as hashing on the grouping attribute, ensuring all tuples of a group reach the same worker. Each worker can then pipeline local aggregation and emit group results once its partition is complete.

Hence shuffle-based grouping is interactive, though it may be constrained by available cores and group metadata. In addition, the initial exchange can be a dominating cost factor.

2.2.4 ORDER BY. An ORDER BY clause is inherently blocking: producing the final sorted output requires reading all input tuples. Formally, its inception point is bounded by the completion of its producer p , i.e.,

$$ip_{order} \geq cp_p.$$

However, ORDER BY can be interactive when tuples arrive sorted on the desired attribute from the upstream operator. This occurs, for example, with index scans or with grouped aggregations that preserve order on the grouping attribute. In these cases, results can be streamed immediately while maintaining the global sort order, making the operator interactive i.e.

$$ip_{order} \geq ip_p.$$

By contrast, queries requiring sorting on derived attributes - whose values or orderings are not known a priori - cannot currently be evaluated interactively.

2.2.5 Duplicate Elimination. Duplicate elimination is typically implemented through blocking approaches like sorting or hashing [13]. However, it can be made interactive if the system maintains a data structure (e.g., a hash set) to record tuples that have already been seen; each new tuple can then be checked for membership and emitted immediately if it is distinct. Also, if the input is ordered such that all occurrences of a value appear contiguously - for instance, through an index scan or a shuffle on the distinct key - each distinct value can be produced as soon as its group is complete. In both cases, the operator's inception point precedes the completion of its input, given by

$$ip_{distinct} \geq \min_g cp_g < cp_p.$$

where g ranges over groups of tuples sharing the same distinct key. Distinct-qualified aggregates such as SUM(DISTINCT) follow the same ip and cp characteristics as normal aggregates discussed in Section 2.2.3.

From Operators to Plans. The preceding analysis is at the operator level: each operator can be classified by whether its inception precedes its inputs' completion. In this work, we focus on designing plans that are interactive throughout, i.e. every operator in the interactive plan is interactive. Furthermore, we ensure that the upstream operators provide the conditions required for downstream operators to behave interactively. For instance, an interactive aggregation requires inputs to arrive in a sorted order, which in turn demands an order-preserving scan (e.g., scan on primary or secondary index) and order-preserving joins such as NLJ or INLJ. In the next subsection, we formalize these choices in a rule-based framework for constructing interactive plans.

3 GENERATING INTERACTIVE PLANS

This section explains how to construct an interactive execution plan. We introduce the notion of a fulcrum - an attribute chosen so that the plan begins with an index scan on that attribute. We then explain why we always construct interactive plans as left-deep tree plans. We then craft a set of rules that allow us to generate an

interactive plan given a SQL query and the underlying database optimizer.

Fulcrum. A fulcrum is an attribute, which when scanned sequentially in sorted order (e.g., by index), allows plans to become interactive. When we start the interactive plan with the ordered index scan on the fulcrum attribute, which gives us an order of tuples that becomes useful for interactive GROUP BY/ORDER BY operations, as we see in the following examples.

In Listing 1, we choose *o_orderdate* as the fulcrum. Using an index scan on *o_orderdate* as the first operation of the interactive plan allows us to use index-based grouping for aggregating the sum on *o_o_totalprice* later, making the plan interactive.

Listing 1: GROUP BY Example <pre> SELECT o.o_orderdate, SUM(o.o_totalprice) FROM orders AS o GROUP BY o.o_orderdate;</pre>	Listing 2: ORDER BY Example <pre> SELECT o.shippriority, o.o_orderdate, FROM orders AS o ORDER BY o.shippriority;</pre>
---	--

In Listing 2, we choose *o_shippriority* as the fulcrum, and we start the interactive plan with an index scan on *o_shippriority*, which lends a natural order to the answer tuples outputted. As tuples arrive already sorted on *o_shippriority*, we can answer the query interactively.

The fulcrum attribute also helps keeping track of the answers outputted already, which becomes useful in coordinating with the blocking execution as we will see in Section 4.4. We formally define the fulcrum below:

Definition 3.1 (Fulcrum). The *fulcrum* of an interactive plan is the attribute chosen so that the plan begins with an index scan on that attribute; the interactive execution produces answers in its order.

Note that, we can have multiple fulcrum candidates for some queries. For example, if in the first example, we change the query to have a GROUP BY on both *o_orderdate* and *o_shippriority*, then either attribute can be chosen as fulcrum. We can also choose a function of an attribute (e.g. `extract(year from o_orderdate)`) if functional indexes are supported by the underlying database system. Determining the most efficient fulcrum becomes a challenge. We explain next how a left deep tree plan helps us with the latter challenge, and we discuss how to select the most efficient fulcrum in Section 3.1.

Why Left-Deep Tree Plans? A left-deep (join) tree is a binary tree where each join has its left child as either a base relation or another join, and its right child as a base relation. This structure enables interactive execution, provided that we do not need to materialize any input completely. Also, if the query or downstream operators require a sorted order on the fulcrum, we can provide it naturally, given that we start with an index scan on the fulcrum. In contrast, bushy join plans allow joins between arbitrary subtrees, increasing flexibility in join ordering and parallelism but hindering interactive execution. Even if we have a sorted order on the fulcrum in one of the subtrees, guaranteeing that this order is upheld by the other joins in the tree becomes difficult. For example, consider joining (*Orders* ⋈ *Lineitem*) and (*Customer* ⋈ *Nation*) in a bushy shape

where we have a sorted fulcrum on *Orders*. Even if the join of (*Orders* ⋈ *Lineitem*) maintains a sorted order on the fulcrum, we cannot easily guarantee that (*Customer* ⋈ *Nation*) produces tuples in a way that respects this order.

3.1 Rules for Creating Interactive Plan

Now that it is motivated that interactive plans are left-deep trees, there are two central challenges in formulating such a plan:

- (1) Determining the fulcrum of the plan
- (2) Determining the order of operators in the interactive plan

Both challenges are inter-related – the choice of fulcrum dictates how many index scans we will have and strongly influences the join order. In this work, we take a *rule-based* approach to formulating interactive plans, with rules that can be naturally integrated into existing optimizer rule frameworks [5, 6]. In a typical optimizer workflow, a query is first translated into a logical plan, which is then rewritten into an optimized logical plan (e.g., via predicate push-down, projection introduction, subquery decorrelation etc.). The optimized logical plan is subsequently transformed into a physical plan. SMARTRABBIT intervenes at this stage: it takes the optimized logical plan as input and produces a physical interactive plan.

Table 1 summarizes the rules used to construct interactive plans. The rules are executed in order of their group identifiers, and the process iterates until the plan converges. Rule Group 1 (Fulcrum Selection) identifies the fulcrum. If the query specifies an ORDER BY, the first attribute in that clause is chosen as the fulcrum to ensure results are produced in the requested order. Otherwise, a candidate set is formed from base attributes projected in the query. Note that if the query has a grouping clause, then these base attributes will automatically be a part of the grouping keys. Attributes without an index are discarded, since they cannot provide a sorted access path. If there are no such fulcrum attributes, then we simply move to Rule 1.6 for the optimizer to create a non-interactive plan. From the remaining set, we remove attributes without filters, as early filtering reduces work in the interactive plan. Finally, if both primary- and secondary-indexed attributes remain, we prefer primaries, since secondary indexes often require additional primary probes to fetch missing attributes [13].

Rule Group 2 (Plan Shape) enforces a left-deep tree structure. We keep any GROUP BY or ORDER BY operator at the top of the tree.

Rule Group 3 (Joins) restricts joins to Nested Loop Join (NLJ) or Index Nested Loop Join (INLJ), always preferring INLJ. As we had mentioned in Section 2.2.2, NLJ and INLJ are both interactive join implementations, with INLJ being more efficient as each probing tuple can be resolved immediately with an index lookup, guaranteeing earlier output than a full inner scan. Once the fulcrum is fixed, join ordering is delegated to the optimizer; If we have multiple fulcrum candidates after applying Rule Group 1, interactive plans are enumerated and costed for each, with the least-cost plan implicitly determining the fulcrum.

Rule Group 4 (Aggregation) mandates the use of the SMARTRABBIT aggregation operator for aggregations. We will describe this operator implementation in Section 4.1 in more detail.

Rule Group 5 (Ordering) similarly prescribes the SMARTRABBIT ordering operator for queries with ORDER BY but no grouping.

Table 1: Interactive Plan Rules

Rule Group	No.	Rule
Fulcrum Selection	1.1	If the query has an ORDER BY clause, choose the first attribute of the ORDER BY clause as the fulcrum.
	1.2	If Rule 1.1 is not applicable, construct a fulcrum candidate set from the base attributes projected in the query.
	1.3	Among candidate attributes, prune those that do not have an index defined on them.
	1.4	Among the remaining candidate attributes, prefer those that have a filter condition.
	1.5	Among the remaining candidates, prefer those with primary indexes.
	1.6	If no viable fulcrum candidate exists, delegate plan construction to the underlying optimizer.
Plan Shape	2.1	Always construct the interactive plan as a left-deep tree that begins at the fulcrum index scan, adding all joins as right children along the spine. Place any GROUP BY or ORDER BY operator at the top of the spine.
Joins	3.1	Restrict joins to Nested Loop Join (NLJ) or Index Nested Loop Join (INLJ).
	3.2	If both NLJ and INLJ are possible, choose INLJ.
	3.3	Once the fulcrum is fixed, delegate join ordering to the database optimizer. If the fulcrum is not fixed, enumerate and cost interactive plans for each candidate and select the least-cost plan, which implicitly determines the fulcrum.
Aggregation	4.1	Use the SMARTRABBIT aggregation operator to perform both local and global aggregations.
Ordering	5.1	Use the SMARTRABBIT ordering operator to answer ORDER BY queries without grouping.

Example to show rules in action. Consider the query in Listing 3 that has three tables, four filter conditions and three grouping attributes. The base attributes projected in the SELECT clause give us three initial fulcrum candidates: *l.l_orderkey*, *o.o_orderdate*, and *o.o_shippriority*. Since the query does not contain an ORDER BY clause, Rule 1.1 does not apply. All three attributes have indexes defined, so no candidate is eliminated under Rule 1.2. Next, we examine filters: both *l.l_orderkey* and *o.o_orderdate* appear in filter conditions, while *o.o_shippriority* does not. Following Rule 1.3, we prune *o.o_shippriority*. This leaves us with two candidates: *l.l_orderkey*, which has a primary index, and *o.o_orderdate*, which has only a secondary index. According to Rule 1.4, we prefer the primary, so *l.l_orderkey* is chosen as the fulcrum.

Listing 3: Example TPC-H Query for Rules Framework

```

SELECT l.l_orderkey, o.o_orderdate, o.o_shippriority,
       SUM(l.l_extendedprice*(1-l.l_discount)) AS revenue
FROM lineitem AS l
JOIN orders AS o ON l.l_orderkey = o.o_orderkey
JOIN customer AS c ON o.o_custkey = c.c_custkey
WHERE c.c_mktsegment = 'BUILDING'
      AND o.o_orderdate < DATE '1995-03-15'
      AND l.l_shipdate > DATE '1995-03-15'
      AND l.l_orderkey > 300000
GROUP BY l.l_orderkey, o.o_orderdate, o.o_shippriority;

```

With the fulcrum fixed, the plan shape follows the left-deep tree mandated by Rule Group 2. The join order is effectively determined by schema constraints: Lineitem can only join with Orders, and then Orders with Customer. For these joins, Rule Group 3 prescribes index nested loop joins whenever possible, which applies here since both Orders and Customer have primary indexes on their join keys. Finally, Rule Group 4 ensures that aggregation is handled by the interactive SMARTRABBIT group-by operator, which incrementally produces local and global aggregates.

In Figure 2a, we see the plan chosen by SMARTRABBIT for Listing 3. We also show the alternate plan if we had chosen *o.o_orderdate* as our fulcrum in Figure 2b. We can see that the alternate plan has an extra index scan, hence explaining our rationale behind Rule 1.5.

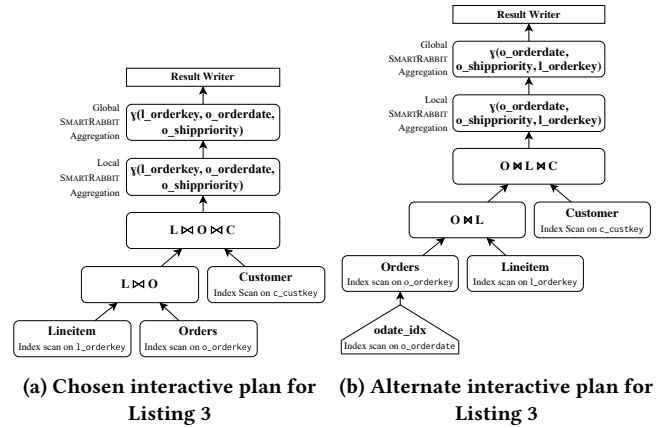


Figure 2: Interactive plans created by SMARTRABBIT.

4 SMARTRABBIT IMPLEMENTATION

In this section, we describe the implementation of SMARTRABBIT in Apache AsterixDB, a scalable open-source [31] database system for managing large volumes of semi-structured and structured data across clusters of machines. AsterixDB's query execution layer is built on Hyracks [8] a dataflow engine for parallel computation. Hyracks supports an operator framework, entitled Algebricks, that implement the necessary operators to support SQL. In AsterixDB, data is distributed over nodes (using hashing on the primary key) and each node contains a predefined number of partitions. As a default, AsterixDB's query processing parallelizes over partitions. Although our implementation is embedded within the AsterixDB code base, the hybrid execution architecture itself is general and can be adopted in other distributed database systems. We chose AsterixDB because it is a widely used, open-source, cluster-based data management system designed for large-scale analytics—an environment where interactive processing is particularly desirable. AsterixDB's scalability is well established, and it has been adopted as an analytics engine for Couchbase [16].

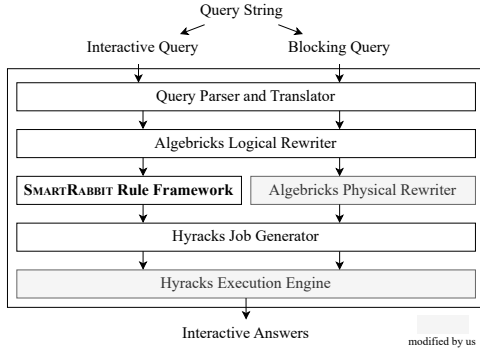


Figure 3: SMARTRABBIT Architecture Diagram

Supporting Hybrid Query Processing. Figure 3 shows how SMARTRABBIT is integrated with AsterixDB and how it can be used to answer queries interactively. When a query arrives, two semantically equivalent copies of it are created - one interactive and one blocking. Both are parsed and translated into Algebricks [7] logical plans. These plans are further optimized by the Algebricks Logical Rewriter. Then the optimized logical plan for the interactive query is transformed into an interactive physical plan by the SMARTRABBIT Rule Framework introduced in Section 3. The blocking plan is generated by the Algebricks Physical Rewriter, which has been slightly modified by us to add new operators (Section 4.4) and modify existing operators that would communicate with the interactive execution. Both plans are converted into Hyracks jobs and are submitted to Hyracks for execution. The user receives answers interactively initially from the interactive execution, and then from the blocking execution once the blocking execution is ready to produce all answers. To support hybrid execution, we modified the Hyracks engine to support simultaneous execution of both interactive and blocking plans. In particular, we added support for localized sorting in different physical operators (Section 4.1), introduced fulcrum informed flushing for interactive execution (Section 4.2) and added mechanisms for the two executions to communicate among each other to maintain the correctness of the results (Sections 4.3 and 4.4). We have also enabled the system to work in a multi-partition parallel execution (Section 4.5). We describe each of these changes below.

4.1 Localized Sorting for Interactive Execution

In the interactive execution, operators consume and process data based on the order of the fulcrum attribute. For GROUP BY queries, if the grouping/ordering is exclusively on the fulcrum attribute, we can process incoming tuples as they arrive in these operators. However often queries will have multiple grouping/ordering keys. As an example, consider a query with grouping keys A and B, where A is the fulcrum. For the aggregations to be correct, we need to ensure that tuples are sorted on both A and B. Since they are sorted on A *by design* we just need to locally sort tuples sharing the same A value on B. Additionally, sorting records sharing the same fulcrum values on a join key/primary key is useful to optimize random accesses during index nested loop joins and primary key lookup. To support these use-cases, we introduce a lightweight *localized sort* operator that performs just enough local (within the same execution partition) reordering on demand. Given a fulcrum

attribute f and an additional order $\vec{o} = (o_1, o_2, \dots, o_k)$ over one or more non-fulcrum attributes, the operator maintains a buffer B of tuples for the current fulcrum value. When the first tuple arrives, f is stored in *currentKey* and the tuple is placed into B . For each subsequent tuple:

- If its fulcrum value equals *currentKey*, it is appended to B .
- else, sort B by $\vec{o}$, emit all tuples to the next operator, reset B to contain the new tuple, and update *currentKey*.

After all input is consumed, any remaining tuples in B are sorted and emitted. The full algorithm is outlined in Algorithm 1. We describe next how the local sorting operator is used to implement aggregation, sorting, and speed up primary index scan operators.

Algorithm 1 Localized Sort (*LSort*)

Require: Stream of tuples T , fulcrum attribute position f , secondary order $\vec{p}$

```

1: Initialize currentKey  $\leftarrow$  null
2: Initialize empty buffer  $B$ 
3: for each tuple  $t \in T$  do
4:   if currentKey = null then
5:     currentKey  $\leftarrow$   $t[f]$ 
6:   else if  $t[f] \neq \textit{currentKey}$  then
7:     Sort  $B$  by  $\vec{p}$ 
8:     Emit  $B$ 
9:     Reset  $B \leftarrow \emptyset$ 
10:    currentKey  $\leftarrow$   $t[f]$ 
11:   end if
12:   Append  $t$  to  $B$ 
13: end for
14: if  $B \neq \emptyset$  then
15:   Sort  $B$  by  $\vec{p}$ 
16:   Emit  $B$ 
17: end if

```

4.1.1 SMARTRABBIT Aggregation. For our discussion of the aggregation implementation, we assume that aggregation happens at two levels: local aggregation and global aggregation, as is typical of distributed systems [30]. The local aggregation is a partial aggregation performed at each partition. These partial aggregates are then merged centrally in the global aggregation operation.

To make aggregation interactive, we exploit the fact that we do not need further sorting if incoming tuples are already sorted on the grouping keys. mechanism. We can simply check for when a different group starts and output the aggregated value. Thus when the grouping key coincides with the fulcrum, aggregation can proceed directly. When the grouping keys include additional non-fulcrum attributes, we apply a localized sort within each fulcrum group to ensure the correct order before aggregation. Note that localized sorting is only necessary for local aggregations. At the global level, all local aggregators have already ensured that all tuples are sorted on the entire grouping key combination. For local aggregation emits one group at a time as subgroups complete, while global aggregation merges these partial groups on arrival, avoiding any additional sorting or blocking. The algorithm for SMARTRABBIT aggregation is outlined in Algorithm 2.

Algorithm 2 SMARTRABBIT Aggregation

Require: Stream T sorted by fulcrum f ; grouping key G ; aggregate family $\mathcal{A}$

```

1:  $R \leftarrow G \setminus \{f\}$ ;  $currentF \leftarrow \perp$ ;  $Block \leftarrow []$ 
2: function PROCESSBLOCK( $F$ ,  $Block$ )
3:   if  $R \neq \emptyset$  then
4:     sort  $Block$  by  $R$  ▷ localized sort within fulcrum  $F$ 
5:   end if
6:    $lastK \leftarrow \perp$ ;  $state \leftarrow \perp$ 
7:   for each tuple  $u$  in  $Block$  (in current order) do
8:      $k \leftarrow \pi_G(u)$ 
9:     if  $lastK \neq \perp$  and  $k \neq lastK$  then
10:      emit ( $lastK$ ,  $FINALIZE_{\mathcal{A}}(state)$ )
11:       $state \leftarrow \perp$ 
12:    end if
13:     $state \leftarrow ACCUMULATE_{\mathcal{A}}(state, u)$ ;  $lastK \leftarrow k$ 
14:  end for
15:  if  $lastK \neq \perp$  then
16:    emit ( $lastK$ ,  $FINALIZE_{\mathcal{A}}(state)$ )
17:  end if
18: end function
19: for each tuple  $t \in T$  do
20:   if  $currentF = \perp$  then
21:     $currentF \leftarrow t.f$ 
22:   end if
23:   if  $t.f \neq currentF$  then
24:    PROCESSBLOCK( $currentF$ ,  $Block$ );  $Block \leftarrow []$ ;  $currentF \leftarrow t.f$ 
25:   end if
26:    $Block.APPEND(t)$ 
27: end for
28: PROCESSBLOCK( $currentF$ ,  $Block$ ) ▷ flush final fulcrum block

```

4.1.2 SMARTRABBIT Ordering Operator. Our approach naturally extends to queries with ordering. When a query includes both ORDER BY and GROUP BY, we align the grouping keys with the ordering attributes so that the aggregation output is already produced in sorted order – leveraging the commutativity of grouping. For queries with only ORDER BY, we use the SMARTRABBIT Ordering Operator that uses a localized sort to incrementally sort tuples per fulcrum on the remaining ordering attributes to deliver the desired order.

4.1.3 Optimizing Primary Index Scan. We found that the interactive plan sometimes may require multiple index nested loop joins to join all the relations and maintain the sorted order of the fulcrum attribute. These joins often use secondary indexes to probe inner relations and we may need to further probe primary indexes to obtain the attributes required for the downstream operators. This extra scan can be expensive, as we may randomly access a large primary index multiple times to even emit a single output tuple. To optimize both the primary key lookup, we can use the Localized Sort to additionally sort all tuples sharing the same fulcrum value on the primary index value. Once tuples are partially clustered on the primary index value, the cost of random I/Os in the interactive plan reduces drastically and the interactive execution can be more effective.

4.2 Fulcrum-Informed Flushing

In conventional push-based query engines, operators propagate data downstream once their internal buffers (or frames) are filled

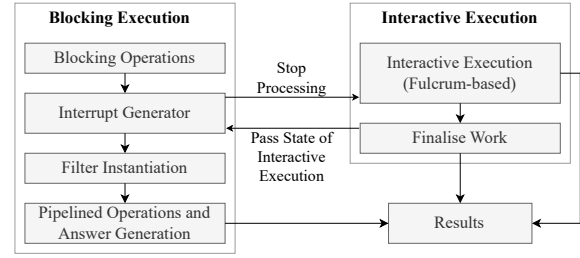


Figure 4: Interrupt Logic and Handover Filter

[26]. For instance, in AsterixDB, tuples are appended to a *frame* until it reaches its capacity, at which point the frame is flushed to the next operator [7]. While this policy ensures high throughput, it also introduces latency for an interactive setting—especially when we want to expedite tuples belonging to distinct key ranges or join anchors are co-located within the same frame. SMARTRABBIT departs from this purely size-driven flushing strategy through a mechanism we term as *fulcrum-informed flushing*. Each physical operator in SMARTRABBIT is made aware of the current fulcrum value during query planning. As tuples are produced, the operator monitors changes in the fulcrum value; if a new fulcrum value is encountered, the operator proactively flushes any partially filled frame and starts a new one. This allows downstream operators to begin processing results for the completed fulcrum immediately, without waiting for the frame to fill. This approach maintains the push-based execution semantics of AsterixDB while introducing adaptive, content-aware flushing that aligns operator boundaries with the logical structure of the interactive plan.

4.3 Communicating Between Executions

As discussed earlier, in hybrid processing, we need to stop the interactive execution when the blocking execution is ready to produce answers. This is achieved by adding a "signaling logic" to specific operators in the blocking and interactive plans. To maximize interactivity, an interrupt (by signaling logic) is inserted after all the blocking operations of the blocking plan have completed. For example, Figure 4, shows that the blocking execution sends an interrupt to the interactive execution after the rest of the blocking plan is complete and only pipelined (interactive) operations remain.

The interactive execution periodically checks for this signal; once detected, it stops processing safely after emitting its last answer and sends an acknowledgment back to the blocking execution, which then resumes its operation. As our experiment results show, this handover happens instantly and does not add overhead to our latency. Further, if the interactive execution completes prior to the blocking execution (which can happen for some queries), a signal is sent from the interactive execution to the blocking execution to stop the query execution since the interactive execution has already produced all the answers and the blocking execution can stop freeing up resources.

4.4 Handover Filter

Once the interactive execution has stopped, the blocking execution must avoid re-emitting the answers already produced. To do so, a handover filter is added to the blocking plan. During planning phase, we add an empty filter (i.e., a condition that always

resolves to true) to the blocking plan at the planning stage and, furthermore, we ensure that the filter is not pushed down the operator tree by the optimizer. When the interactive plan finishes its execution (as a result of an interrupt from the blocking plan), it communicates the state of its execution (i.e., the last fulcrum value, and other non-fulcrum values in case the query consisted of multiple ordering and/or grouping attributes, that it has fully processed), to the blocking plan. These values, written to a shared buffer between the interactive and blocking plan, enable the blocking plan to dynamically learn the filter value at runtime (Figure 4) which empowers the blocking plan to not emit any answers already emitted by the interactive plan. For example, if the fulcrum is *custkey* and the last interactive answer has *custkey* = 1000, the blocking execution learns the filter *custkey* > 1000. This guarantees complete coverage while preventing overlap. For queries with composite grouping keys, the learned condition extends naturally to a conjunction over all grouping attributes. For instance, if the grouping keys are (*custkey*, *orderdate*) and the last interactive answer has (*custkey*, *orderdate*) = (1000, '1995-03-15'), the blocking execution learns the filter (*custkey* > 1000) OR (*custkey* = 1000 AND *orderdate* > '1995-03-15'). We emphasize that the sort order on the fulcrum offers a simple and efficient way to avoid duplications compared to the typical duplicate eliminations that are blocking.

4.5 Challenges for Parallel Execution

When executing interactive query plans in a distributed parallel system, we need to solve two challenges: (i) How can we retain sorted orders? And (ii) How can we communicate the handover filter value?

4.5.1 Maintaining a sorted order of inputs. The success of interactive execution relies on preserving the sorted order on the fulcrum attribute. This enables us to answer one fulcrum value at a time and to safely hand over query processing to the blocking execution when needed. When all data resides in a single partition, maintaining this order is straightforward: each operator receives data from a single input source, and the order is naturally propagated downstream. In contrast, in a distributed setting, tuples with the same fulcrum value may reside across multiple nodes. When these nodes send locally sorted streams to a common receiver, their interleaving can violate the global sorted order, complicating the downstream operators like aggregation.

As an example, suppose we propagate (*a*, *b*) tuples from four senders to a single receiver, where each sender locally orders its output by attribute *a*. Senders 1 and 2 are the fastest and send tuples (1, 6) and (1, 5) first. Before senders 3 and 4 transmit their tuples with *a* = 1, sender 2 sends (2, 7) – thereby breaking the global sorted order at the receiver. This becomes problematic for downstream interactive operators such as GROUP BY and ORDER BY, which depend on receiving sorted input and would otherwise produce incorrect results.

To address this, we adopt a pull-based merge strategy using merge queues (e.g., heaps). Similar to distributed merge sort, each receiver maintains a sorted priority queue that pulls tuples from sender queues in global sorted order.

4.5.2 Communicating the value of the handover filter. In a parallel system, the result writer operator—the operator responsible for emitting results to the user—is itself parallel, i.e., multiple writer instances run on different partitions. This parallelism makes it impossible to maintain a global sorted order across all writers. If the query does not require ordered output, results could in principle be returned in any order. However, ensuring correctness of the handover filter becomes problematic: when the blocking execution interrupts the interactive plan, the last tuple emitted by a fast partition may not correspond to the true global handover point. A faster partition may have already produced tuples with larger fulcrum values, while a slower partition has yet to emit tuples with smaller fulcrum values.

To guarantee both correctness and consistent termination, we execute the result writer of the interactive plan as a single (unpartitioned) instance. This preserves the global sorted order, provides a well-defined final tuple for the handover filter, and avoids race conditions across partitions. The trade-off is reduced parallelism at the result stage, but this impacts only the final emission operator, not the rest of the interactive pipeline.

5 EVALUATION

In this section, we evaluate the performance of SMARTRABBIT. SMARTRABBIT Section 5.1 introduces the metrics used to evaluate SMARTRABBIT. Experimental setup is discussed in Section 5.2. Section 5.3 describes the experimental study. Lastly, we explain performance trends using performance profiling in Section 5.4.

5.1 Metrics

We use the following three metrics to evaluate SMARTRABBIT against other approaches.

Time To First Results (TTFR) [19] that measures the time when the first results of the query are produced since its start. TTFR has been commonly used in prior work to quantify interactivity.

Interactivity Fraction (IF) that measures the fraction of answers SMARTRABBIT returns with its interactive execution, before the blocking execution takes over.

End-to-End Latency that measures the time when the query terminates and all its results have been computed.

The first two measures – TTFR and IF – together provide a holistic picture of the degree of interactivity by indicating how fast and what fraction of results will SMARTRABBIT deliver to downstream operators/users. The latency measure will be used to compare the potential overhead/increase in time compared to the standalone blocking approach to ascertain the cost of interactivity that SMARTRABBIT supports.

5.2 Experimental Setup

Datasets We use TPC-H [32] benchmark for our experiments. We evaluate SMARTRABBIT on five different scale factors (SFs): 10, 30, 50, 100, and 300.

Queries We select seven TPC-H queries for our experiments. We do not include queries that result in a single answer (since interactivity would not make a difference to such queries), and also do not include queries for which the optimizer chooses a non-blocking

plan - such queries are already interactive. For experiments in Section 5.3.1, Section 5.3.3 and Section 5.3.4, we removed the LIMIT clause from the queries as we wanted to focus on the performance of SMARTRABBIT when a large number of results are returned. We separately compare the performance of SMARTRABBIT with a pure blocking approach for LIMIT queries in Section 5.3.2. A longer discussion of queries used is presented in the appendix.

System Configuration Experiments were conducted in AsterixDB instances (version 0.9.9) hosted on a server equipped with 4× Intel Xeon E5-4640 v2 CPUs (40 physical cores, 80 hardware threads) at 2.20 GHz and 125 GiB RAM, with a 7.68 TB Samsung PM893 SATA SSD running Ubuntu 24.04.2.

Parallelism of System We evaluated SMARTRABBIT under three representative configurations of AsterixDB: (i) 4 nodes with 1 partition per node, (ii) 4 nodes with 2 partitions per node, and (iii) 4 nodes with 5 partitions per node. This range of configurations allowed us to study the effectiveness of our approach across varying degrees of intra-node parallelism and data partitioning granularity. The default setting for our experiments is (i) above.

Memory We set the memory of our Asterix DB instances to a default of 1 GB for experiments in Section 5.3.1, Section 5.3.2, Section 5.3.3, Section 5.3.4 as we want to compare our approach across scale factors and partitions generally when all the data does not reside in the Asterix DB cache. We then also evaluate the performance of our system in a cache resident setting in Section 5.3.7.

Number of Runs We report the average over 10 runs with one upfront warm-up run to exclude cold-start effects.

5.3 Experimental Results

5.3.1 SMARTRABBIT vs Blocking Execution. Figure 5 shows the latency as well as TTFR of the queries run with SMARTRABBIT compared to the queries run based on the original blocking approach generated by the AsterixDB optimizer. TTFR for a blocking approach (not shown in the figure) is the same as its latency since the plan only provides all the answers at the end of the execution. For queries Q3, Q8, Q9 and Q10, the TTFR for SMARTRABBIT is negligible (over 2 orders of magnitude less, while that for Q1, Q4, Q12 is still over 20% reduction compared to the blocking strategy. The improvement in the latter query set is limited since these queries produce a small number (≤ 5) of results that are generated later during the interactive execution. The maximum latency incurred by SMARTRABBIT compared to the blocking execution is less than 4.5% which demonstrates that SMARTRABBIT does not add significant overhead to the original blocking plan. Interestingly, unexpectedly, SMARTRABBIT achieves better latency compared to the original blocking strategy in 4 out of the 7 queries. The reason for this is investigated in Section 5.4. Finally, SMARTRABBIT achieves IF values between 8-60% (Figure 6) demonstrating its ability to achieve interactivity.

5.3.2 Performance on LIMIT queries. We next explore the use of SMARTRABBIT for LIMIT queries which, as discussed in the introduction, could be used as an alternate way to support interactivity during query processing. In particular, three representative TPC-H queries—Q3, Q9, and Q10 — were used to study the behavior of queries with LIMIT clauses. We executed these three queries over TPC-H SF 300 in AsterixDB by themselves and compared that to

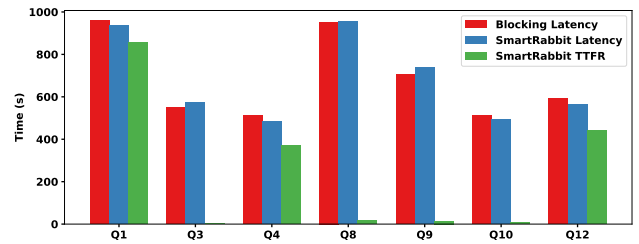


Figure 5: SmartRabbit vs. Blocking: Latency and TTFR for all queries in SF100

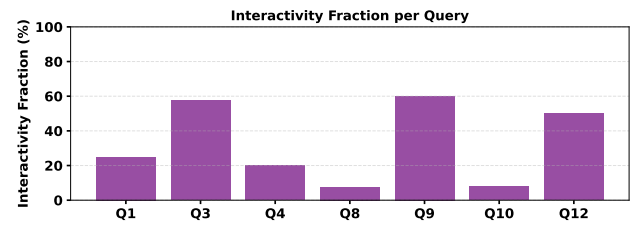


Figure 6: IF of SMARTRABBIT across all queries

SMARTRABBIT wherein we stop the execution as soon as it finds the required number of answers specified in the LIMIT. We note that the time taken by AsterixDB for these queries is essentially the same it would take without LIMIT. The queries contain a GROUP BY which prevents the optimizer to push the LIMIT clause down the query tree. Figure 7 shows the execution times of SMARTRABBIT for varying sizes of LIMIT.

The curves remains nearly flat and close to zero for a wide range of limits and even LIMIT 10 000 tuples are returned within a few seconds - i.e. upto 500x faster than the blocking approach. This indicates that SMARTRABBIT can produce small to moderately large subsets of answers very quickly. Only near the end – when the LIMIT approaches the total answer cardinality – does the curve rises. This experiment highlights an important additional benefit of SMARTRABBIT: beyond delivering interactive results, it can also **serves as a practical approach for efficiently implementing LIMIT queries in modern database systems.**

5.3.3 Comparison across different scale factors. Figure 8 shows a comparison of the average blocking execution latency, SMARTRABBIT latency and TTFR across scale factors. As expected, the latencies increase with increasing scale factor. SMARTRABBIT, on an average, takes slightly less time as compared to the blocking execution in all the scale factors. We divided our queries into two groups: the first group (Q1, Q4 and Q12) produce less than five results and the number of answers remain constant across all scale factors. All the queries in the second group (Q3, Q8, Q9 and Q10) produce a large number of results, with the number of results increasing as scale factor increases. We plot the average TTFR of each group separately as the two groups exhibit different patterns. For the first group, TTFR rises with scale factor but stays well below the blocking latency. For the second group, we note that the **TTFR is negligible even at the largest SFs.** This shows that SMARTRABBIT's interactive execution can start producing answers in mere

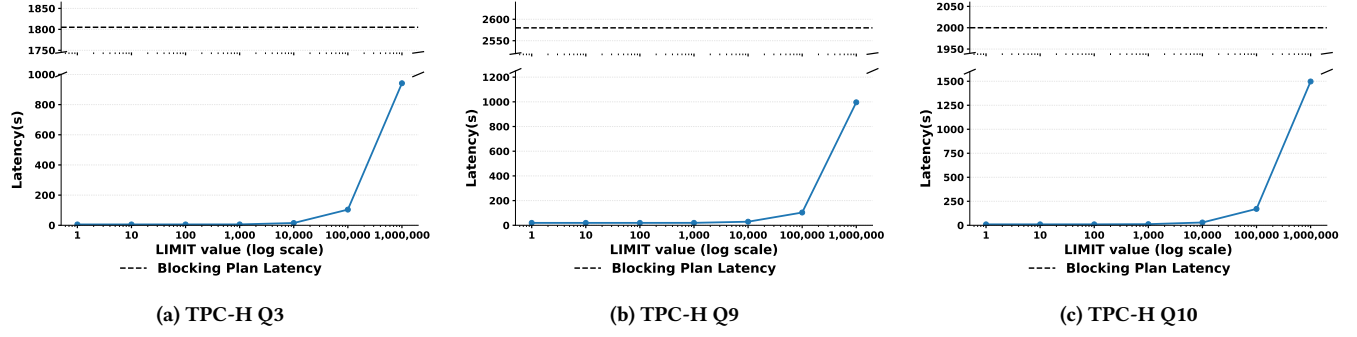


Figure 7: Performance of SMARTRABBIT on LIMIT queries.

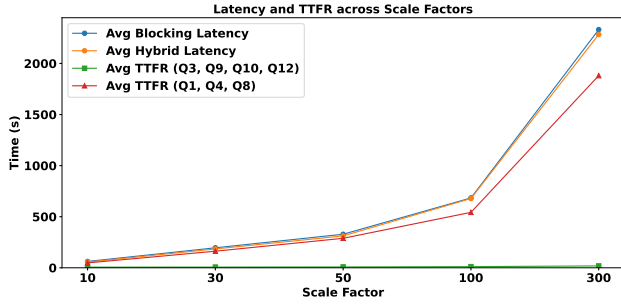


Figure 8: SmartRabbit vs. Blocking: Latency and TTFR across all SFs

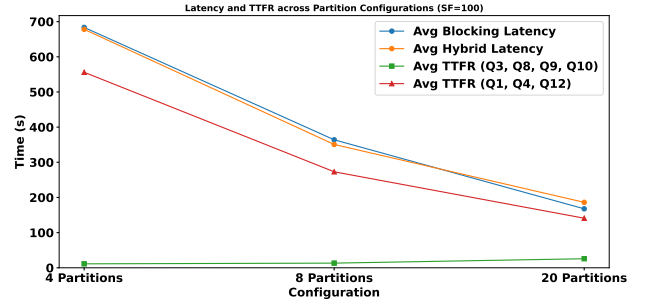


Figure 10: SmartRabbit vs. Blocking: Latency and TTFR across different configurations

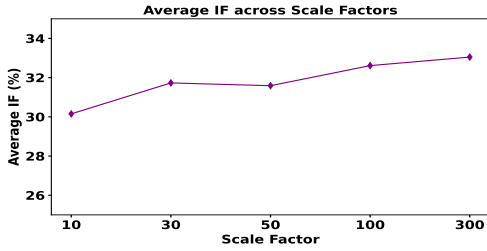


Figure 9: Average IF of SMARTRABBIT across SFs

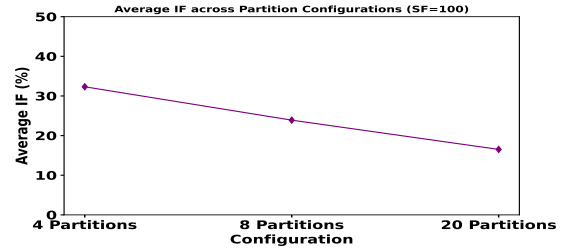


Figure 11: Average IF of SMARTRABBIT across different configurations

seconds even for large scale factors, whereas the blocking time can take upto 30 minutes to produce even one answer.

Figure 9 shows the average IF across the five scale factors. We can see that the average IF follows a gradual upward trend. This shows that SmartRabbit becomes **even more interactive as the scale factor increases**.

5.3.4 Comparison across different number of partitions. We next explore how SMARTRABBIT performs relative to the blocking execution as we increase the partition count in AsterixDB. For this study, we use TPC-H SF 100 across three AsterixDB configurations with 4, 8, and 20 partitions. Figure 10 shows that the average latency of the blocking and SMARTRABBIT queries across all partitions reduce as we increase the number of partitions. This is expected as the overall performance of the system improves with number of partitions. Only a slight overhead is observed in the SMARTRABBIT latency time in the 20 partition setting. This shows that SMARTRABBIT scales well with partition count in terms of latency. The average TTFR

of the first group of queries (Q1, Q4, Q12) also decreases whereas the average TTFR of the second group slightly increases but still remains at least 10 $\times$ less compared to the latency of the blocking plan. The average IF of SMARTRABBIT shows a modest decline as the number of partitions grows. This is because operations like the single-partition result writer and sorted merge queues require cross-partition coordination that lowers SMARTRABBIT's interactive production rate as partitions increase. Nonetheless, SMARTRABBIT delivers interactive behavior even at higher degrees of parallelism, demonstrating robustness under increasing partition counts.

5.3.5 Performance for SSB Dataset. We consider 9 SSB queries that output multiple answers and plot the interactivity fractions in Figure 13 and the latency in Figure 12. for three scale factors: 10, 30 and 100. We observe very similar performances for 9 queries across all scale factors. This is because all of them have very similar query plans for both the interactive and the blocking plan, and

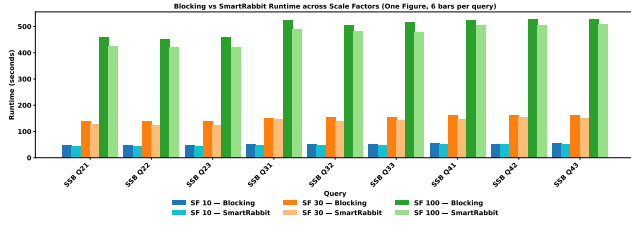


Figure 12: SmartRabbit vs. Blocking: Latency across SFs on SSB

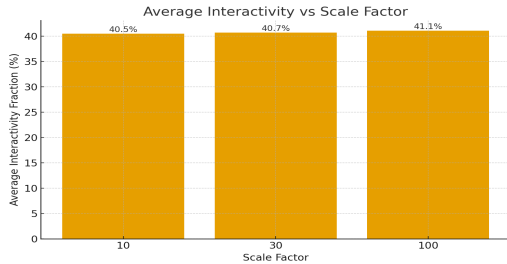


Figure 13: Average IF of SMARTRABBIT across SFs on SSB

the execution cost in both plans is dominated by the Lineorder table that is $75 \times 24,000 \times$ bigger than all the other tables and the interactive plan for all the queries has a fulcrum on the Date table. With respect to latency, we observe that SMARTRABBIT finishes earlier than the pure blocking execution consistently, consistent with the observation in our TPC-H experiments. The TTFR numbers are within a few seconds for all the queries and we do not plot it.

5.3.6 Performance when the OS cache is not in play. We realized that for the experiments so far, the setup is not fully *disk resident* as the underlying OS also caches the data pages that are being accessed. This means that the numbers captured would not fully reflect the true scenario that we try to emulate: i.e. with 1 GB of cache, any data not present in the cache has to be accessed from the disk. So we conducted another set of experiments where we clear the OS cache after every run and we redid the experiments of Section 5.3.3. We notice that the latency of the blocking execution increases by 5-10% across scale factors. The interactive execution is comparatively even faster than the blocking execution, especially in higher SFs. Moreover the average IF at each SF increases. The results are plotted in Figure 15 and Figure 14. This shows that without the support of the OS cache, SMARTRABBIT becomes even more efficient than the blocking execution.

5.3.7 Performance on a fully cache-resident system. We examine system behavior when the entire dataset fits in memory. We use the TPC-H benchmark at SF 10 to ensure all data resides in the Asterix DB cache. We compare this cache-resident setting against the default configuration with a 1 GB cache. As shown in Table 2, end-to-end latency for blocking executions drops by 15-40%, and SMARTRABBIT's latency now closely matches the standalone blocking plan. In contrast, under non-cache-resident conditions, SMARTRABBIT finishes earlier indicating that the concurrent interactive

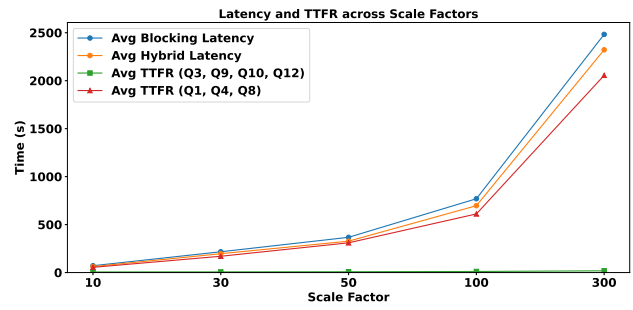


Figure 14: SmartRabbit vs. Blocking: Latency and TTFR across all SFs

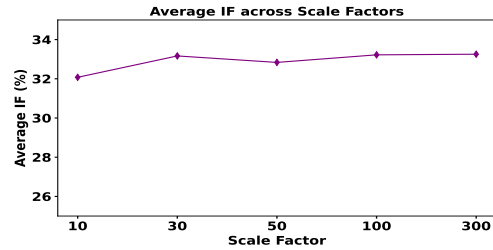


Figure 15: Average IF of SMARTRABBIT across SFs

Table 2: Average execution time and Interactivity Factor (IF) under different cache conditions.

Condition	Blocking (s)	SmartRabbit (s)	IF
Non-Cache-Resident	62.14	56.00	0.304
Cache-Resident	45.85	44.71	0.273

Table 3: Inception points for interactive vs. blocking executions.

Operation	Inception Point (s)	Description	Inception Point (s)
Index Scan on L	0.138	(C $\bowtie$ O) $\bowtie$ L join (HHJ)	45.800
L $\bowtie$ O (INLJ)	0.833	Local Aggr. (sort-based)	53.249
Local Aggr. (SMARTRABBIT)	1.806	Global Aggr. (sort-based)	54.050
Global Aggr. (SMARTRABBIT)	3.855	Result Writer	54.280
Result Writer	4.108		

(a) Connector inception points for interactive execution.

(b) Inception points for blocking execution.

execution warms the cache for the blocking plan. The slight reduction in IF is due to the faster completion— when blocking finishes sooner, the interactive execution has less time to produce early answers.

5.4 Explaining the executions using the profiler

We use the AsterixDB profiler to explain two key observations from our experiments: (i) why the interactive execution begins producing results almost instantly, and (ii) why, at smaller scales, the blocking execution finishes earlier when run concurrently with an interactive execution. All prior experiments were performed with the profiler disabled to avoid overhead; here we enable it selectively for TPC-H Query 3.

Table 4: Key scan and join operators: SMARTRABBIT blocking vs. standalone blocking at SFs 10 and 100 (seconds).

Plan	Scale Factor 10		Scale Factor 100	
	Hybrid	Blocking	Hybrid	Blocking
Duration	50 s	54 s	612 s	589 s
Scan on Orders	3.90	4.30	59.39	52.82
Scan on Lineitem	18.21	24.01	267.68	261.38
$C \bowtie O$	12.03	13.22	21.94	14.53
$(C \bowtie O) \bowtie L$	6.50	5.22	87.15	81.47

Operator Inception Times. Table 3a summarizes the inception points (with respect to query start time) for both plans for Q3. In the interactive plan, most operators start within a few seconds of query submission, consistent with the early arrival of results observed in Figure 1. The `Lineitem` index scan acts as the fulcrum, rapidly streaming frames upward to the join and aggregation operators, all of which begin emitting partial results in quick succession. In contrast, Table 3b shows that the blocking plan is gated by pipeline breakers. Most importantly the first output of the $C \bowtie O \bowtie L$ is only seen at 46.606 seconds, once the entire `Lineitem` dataset has been scanned and partitioned. The inception point of this operation and subsequent operations are delayed because each blocking operator in the blocking plan must fully materialize its input before the next operator can start.

Shared I/O and Cache Effects. At smaller scale factors, the blocking execution gains a modest advantage when executed alongside the interactive execution because it benefits from the latter’s early I/O activity. As shown in Table 4, the SMARTRABBIT-blocking execution completes in 50 s compared to 54 s for the pure blocking run and all the time difference can be attributed to the time required to scan `Lineitem`. Further, when the entire dataset is preloaded in the Asterix DB cache (as in Section 5.3.7), the buffer-resident (in-memory) execution finishes in just 36 s, clearly revealing the additional cost introduced by I/O. However, this effect does not persist at larger scales such as SF 100 (Table 4), where the cost of I/O for scans and join partitioning or probing becomes dominant. At these higher scales, both executions contend for I/O bandwidth, and the benefit of caching diminishes, causing slight overhead for the SMARTRABBIT blocking execution.

6 RELATED WORK

From a motivation perspective, SMARTRABBIT is related to prior work on interactive query processing using LIMIT clauses and on approximate query processing (AQP), as discussed in the introduction. Ilyas et al. explored interactive top-k queries by converting a query into a sequence of queries with LIMIT clause with varying offsets, enabling incremental computation of results [18]. Li et al. proposed the *rankagg* operator to minimize the number of tuples processed for LIMIT queries [21]. These approaches, however, require repeated execution, offer limited opportunities for pushing LIMIT down the query plan (especially in push-based architectures), and apply to a narrow class of queries. AQP also shares a similar goal of improving interactivity but focuses on approximate aggregates with progressively reduced error, using a different technical strategy. In contrast, SMARTRABBIT targets a broader class of queries and supports interactive evaluation of exact answers.

From a technical and methodological perspective, SMARTRABBIT is loosely related to prior work on adaptive query processing and optimization, such as tuple-routing architectures like Eddies [4, 28], which dynamically route tuples to different operators based on the expected execution strategy. A more recent example is Plaque [22], which learns predicates from downstream operators (e.g., aggregations and joins) at runtime and pushes them down dynamically to reduce computation – significantly extending classical static predicate pushdown. SMARTRABBIT shares this adaptive flavor, making flexible decisions during query execution based on runtime state, though its goals and mechanisms differ substantially.

The work most closely related to SMARTRABBIT in technique is progressive query optimization (POP) [23]. POP introduced a CHECK operator into the query plan to detect significant deviations in intermediate cardinalities and trigger re-optimization when beneficial. A key aspect of POP is its ability to reuse already computed results after re-optimization. The idea of POP has also been adopted by DataBricks [35]. While POP focuses on maintaining cost optimality through plan re-optimization, SMARTRABBIT instead coordinates between interactive and blocking execution based on operator readiness, targeting interactivity rather than re-optimization.

An orthogonal but complementary line of work is *incremental query processing* [34], where queries are executed over extended periods with planned pauses. The goal is to process available data early, often during off-peak times, and complete the remaining work near the deadline to improve cost and resource efficiency. Although incremental processing is not directly related to SMARTRABBIT, combining the two approaches could yield synergistic benefits for complex analytical workloads.

Duck DB’s Instant SQL provides live query previews by re-executing partial queries as the user types, leveraging local execution, AST-based rewriting, and caching [25]. While Instant SQL focuses on front-end responsiveness during query formulation, SMARTRABBIT addresses runtime interactivity – delivering early and incremental results during query execution itself.

7 CONCLUSION AND FUTURE WORK

For large datasets, blocking algorithms delay output for minutes before any results appear. With SMARTRABBIT, we proposed a new system that uses interactive query processing to keep emitting early results while the blocking execution gets ready. This way, we can feed downstream processes even for queries that are typically blocking, such as grouping aggregations. We implemented the system on top of AsterixDB and showed how a hybrid execution schema can provide early results while still yielding almost the same end-to-end query latency as typical blocking query plans. We report that SMARTRABBIT can be more than 500x faster than a LIMIT approach on a blocking execution. We see how our current approach improves on higher scale factors, and also maintains interactivity on increasing the number of partitions.

We see several directions for future work. While our system currently processes the entire data set in both the interactive as well as the blocking plan, a data- and cost-aware partitioning could further improve efficiency by splitting the data sets. We can also think of scheduling the blocking and interactive queries intelligently to

minimize resource contention between the two. Another interesting direction is pushing down filters learned from one execution onto the other execution, exploiting the different orders in which the relations are scanned in each execution. SMARTRABBIT can be extended with approximation techniques to deliver interactive approximate results. While using the standard optimizer significantly helped implementing SMARTRABBIT on a real-world database system, making the optimizer aware of interactive query processing promises further efficiency and performance improvements. Lastly, this work is a study of how interactivity can be achieved, and not how it can be utilized. A promising direction is incorporating user studies to better understand and prioritize interactivity needs.

REFERENCES

- [1] S. Acharya, P. B. Gibbons, and V. Poosala. Congressional samples for approximate answering of group-by queries. In *Proceedings of the International Conference on Management of Data (SIGMOD)*, page 487–498. ACM, 2000.
- [2] S. Agarwal, B. Mozafari, A. Panda, H. Milner, S. Madden, and I. Stoica. Blinkdb: queries with bounded errors and bounded response times on very large data. In *Proceedings of the European Conference on Computer Systems (EuroSys)*, page 29–42. ACM, 2013.
- [3] S. Alsabaiee, Y. Altowim, H. Altwaijry, A. Behm, V. R. Borkar, Y. Bu, M. J. Carey, I. Cetindil, M. Cheelangi, K. Faraaz, E. Gabrielova, R. Grover, Z. Heilbron, Y. Kim, C. Li, G. Li, J. M. Ok, N. Onose, P. Pirzadeh, V. J. Tsotras, R. Vernica, J. Wen, and T. Westmann. AsterixDB: A scalable, open source BDMS. *Proc. VLDB Endow.*, 7(14):1905–1916, 2014.
- [4] R. Avnur and J. M. Hellerstein. Eddies: Continuously adaptive query processing. In *Proceedings of the International Conference on Management of Data (SIGMOD)*, pages 261–272. ACM, 2000.
- [5] E. Begoli, J. Camacho-Rodríguez, J. Hyde, M. J. Mior, and D. Lemire. Apache Calcite: A foundational framework for optimized query processing over heterogeneous data sources. In *Proceedings of the International Conference on Management of Data (SIGMOD)*, pages 221–230. ACM, 2018.
- [6] V. Borkar, Y. Bu, E. P. Carman, N. Onose, T. Westmann, P. Pirzadeh, M. J. Carey, and V. J. Tsotras. Algebricks: a data model-agnostic compiler backend for big data languages. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, page 422–433. ACM, 2015.
- [7] V. Borkar, M. Carey, R. Grover, N. Onose, and R. Vernica. Hyracks: A flexible and extensible foundation for data-intensive computing. In *Proceedings of the International Conference on Data Engineering (ICDE)*, pages 1151–1162, 2011.
- [8] V. R. Borkar, M. J. Carey, R. Grover, N. Onose, and R. Vernica. Hyracks: A flexible and extensible foundation for data-intensive computing. In *Proceedings of the International Conference on Data Engineering (ICDE)*, pages 1151–1162, 2011.
- [9] M. J. Carey and D. Kossmann. On saying “enough already!” in SQL. In *Proceedings of the International Conference on Management of Data (SIGMOD)*, pages 219–230. ACM, 1997.
- [10] J. Dittrich, B. Seeger, D. S. Taylor, and P. Widmayer. Progressive merge join: A generic and non-blocking sort-based join algorithm. In *Proceedings of the International Conference on Very Large Data Bases (VLDB)*, pages 299–310. 2002.
- [11] D. Fisher, S. Drucker, and M. Czerwinski. Trust Me, I’m Partially Right: Incremental Visualization Lets Analysts Explore Large Datasets Faster. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*, pages 1673–1682. ACM, 2012.
- [12] G. Graefe. Encapsulation of parallelism in the volcano query processing system. In *Proceedings of the International Conference on Management of Data (SIGMOD)*, pages 102–111, 1990.
- [13] G. Graefe. Query evaluation techniques for large databases. *ACM Computing Surveys*, 25(2):73–170, 1993.
- [14] P. J. Haas and J. M. Hellerstein. Ripple joins for online aggregation. *SIGMOD Rec.*, 28(2):287–298, June 1999.
- [15] J. M. Hellerstein, P. J. Haas, and H. J. Wang. Online aggregation. In *Proceedings of the International Conference on Management of Data (SIGMOD)*, pages 171–182. ACM, 1997.
- [16] M. A. Hubail, A. Alsuliman, M. Blow, M. J. Carey, D. Lychagin, I. Maxon, and T. Westmann. Couchbase Analytics: NoETL for Scalable NoSQL Data Analysis. *Proc. VLDB Endow.*, 12(12):2275–2286, 2019.
- [17] I. F. Ilyas, W. G. Aref, and A. K. Elmagarmid. Supporting top-k join queries in relational databases. *VLDB Journal*, 13(3):207–221, 2004.
- [18] I. F. Ilyas, G. Beskales, and M. A. Soliman. A survey of top-k query processing techniques in relational database systems. *ACM Comput. Surv.*, 40(4):11:1–11:58, 2008.
- [19] Z. G. Ives, D. Florescu, M. Friedman, A. Y. Levy, and D. S. Weld. An adaptive query execution system for data integration. In *Proceedings of the International Conference on Management of Data (SIGMOD)*, pages 299–310. ACM Press, 1999.
- [20] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *Proceedings of International Conference on Machine Learning*, volume 202, pages 20854–20873. PMLR, 2023.
- [21] C. Li, K. C. Chang, and I. F. Ilyas. Supporting ad-hoc ranking aggregates. In *Proceedings of the International Conference on Management of Data (SIGMOD)*, pages 61–72. ACM, 2006.
- [22] Y. Lin and S. Mehrotra. PLAQUE: automated predicate learning at query time. *Proc. ACM Manag. Data*, 2(1):46:1–46:25, 2024.
- [23] V. Markl, V. Raman, D. E. Simmen, G. M. Lohman, and H. Pirahesh. Robust query processing through progressive optimization. In *Proceedings of the International Conference on Management of Data (SIGMOD)*, pages 659–670. ACM, 2004.
- [24] M. F. Mokbel, M. Lu, and W. G. Aref. Hash-merge join: A non-blocking join algorithm for producing fast and early join results. In *Proceedings of the International Conference on Data Engineering (ICDE)*, pages 251–262, USA, 2004. IEEE.
- [25] MotherDuck Team. Instant sql is here: Speedrun ad-hoc queries as you type. <https://motherduck.com/blog/introducing-instant-sql/>, April 2025. Accessed: 2025-10-17.
- [26] T. Neumann. Efficiently compiling efficient query plans for modern hardware. *Proc. VLDB Endow.*, 4(9):539–550, June 2011.
- [27] C. Olston, E. Bortnikov, K. Elmeleegy, F. Junqueira, and B. C. Reed. Interactive analysis of web-scale data. In *Biennial Conference on Innovative Data Systems Research (CIDR)*, *Online Proceedings*. www.cidrdb.org, 2009.
- [28] V. Raman, A. Deshpande, and J. M. Hellerstein. Using state modules for adaptive query processing. In *Proceedings of the International Conference on Data Engineering (ICDE)*, pages 353–364. IEEE Computer Society, 2003.
- [29] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. Access path selection in a relational database management system. In *Proceedings of the International Conference on Management of Data (SIGMOD)*, page 23–34, New York, NY, USA, 1979. ACM.
- [30] A. Shatdal and J. F. Naughton. Adaptive parallel aggregation algorithms. In *Proceedings of the International Conference on Management of Data (SIGMOD)*, page 104–114. ACM, 1995.
- [31] The Apache Software foundation. AsterixDB on GitHub. <https://github.com/apache/asterixdb/>, 2025. Accessed: 2025-10-05.
- [32] Transaction Processing Performance Council (TPC). TPC-H: A decision support benchmark. <https://www.tpc.org/tpch/>, 2025. Accessed: 2025-06-15.
- [33] T. Urhan. *Adaptive query processing for improving responsiveness of wide-area queries*. PhD thesis, USA, 2002.
- [34] Z. Wang, K. Zeng, B. Huang, W. Chen, X. Cui, B. Wang, J. Liu, L. Fan, D. Qu, Z. Hou, T. Guan, C. Li, and J. Zhou. Tempura: a general cost-based optimizer framework for incremental data processing (journal version). *VLDB Journal*, 32(6):1315–1342, 2023.
- [35] M. Xue, Y. Bu, A. Somani, W. Fan, Z. Liu, S. Chen, H. V. Hovell, B. Samwel, M. Mokhtar, R. Korlapati, A. Lam, Y. Ma, V. Ercegovic, J. Li, A. Behm, Y. Li, X. Li, S. Krishnamurthy, A. Shukla, M. Petropoulos, S. Paranjpye, R. Xin, and M. Zaharia. Adaptive and robust query execution for lakehouses at scale. *Proc. VLDB Endow.*, 17(12):3947–3959, 2024.